Introduction to Stata

Lecture 2: Creating variables and combining datasets

Marco Rosso

University of Bologna

A.Y. 2026/27

Where we are, where we go

From Lecture 1 (`data/clean/`)

- **`individuals.dta`**: one row per person-wave
- **`rfam, ricf, cons, peso`**: one row per household-wave
- **`regions.dta`**, **`deflator.dta`**

Missing a file? Run **`L1_solutions.do`**.

Handbook: Chapters 4 and 5; § 2.6; § 12.1–12.2.

By the end of today

- **`shiw_hh.dta`**: one row per household-wave, with income, wealth, consumption, weights and the main earner's characteristics
- **`individuals_full.dta`**, **`region_year.dta`**

PART 1

Demonstration

generate, replace, recode

Handbook § 4.1, 4.3, 4.4

```
generate female = (sex == 2)                        // 1 if true, 0 if false
generate university = (educ >= 5) if !missing(educ) // missing stays missing
generate educ_years = .                             // start empty...
replace  educ_years = 13 if educ == 4               // ...and fill
recode age (min/17 = 1 "0-17") (18/34 = 2 "18-34") ///
          ..., generate(age_group)
```

`generate`	new variable; functions such as `ln()`, `inlist()`, `inrange()` (§ 2.4)
`replace`	change values of an existing variable
`recode`	map ranges or codes to new codes, with labels
`label values x yesno`	one label for all 0/1 dummies

Always check a recoding with a two-way `tabulate` of the old and the new variable.

by-groups: _n, _N and egen

Handbook § 2.6, 4.2

```
bysort year nquest: gen N = _N
assert N == hhsize
egen n_children = total(age < 14), ///
     by(year nquest)
gen tmp = educ_years if cfred == 1
bysort year nquest: egen head_educ = max(tmp)
```

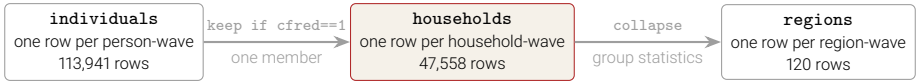
- `_n`: position in the group; `_N`: size of the group
- `egen`: functions over many rows (`total`, `mean`, `max`, ...)

nquest	nord	age	_n	_N	n_ch.
101	1	45	1	3	1
101	2	43	2	3	1
101	3	9	3	3	1
102	1	78	1	1	0
103	1	38	1	2	0
103	2	36	2	2	0

Stylised example, one wave.

Changing the unit of observation

Handbook § 3.8, 5.5



- Household variables (income, wealth) are the same for all members: keep one row per household
- The main income earner (`cfred == 1`) describes the household: `head_age`, `head_educ`, …
- First compute what needs all members (number of children), *then* drop rows

merge: adding columns

Handbook § 5.3

Type	Key unique in	Example
`1:1`	both	household + its income
`m:1`	using only	household + its region
`1:m`	master only	region + its households
`m:m`	never use it	

Master = data in memory; using = file on disk.

Useful options: `keepusing()`, `assert(match)`, `keep(master match)`, `nogenerate`

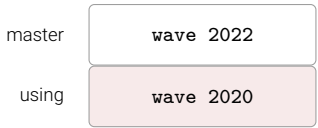
```
merge 1:1 nquest year ///
  using "$clean/rfam.dta", ///
  keepusing(y yl yt ym yc)
tabulate _merge
```

`_merge`	meaning
1	only in master
2	only in using
3	matched

Predict the result of every merge, then check it.

append: adding rows

Handbook § 5.1, 5.2



- `append`: same variables, different observations
- `merge`: same observations, different variables
- `generate(source)` records where each row comes from
- `preserve` / `restore`: snapshot of the data; `tempfile`: a file deleted at the end

reshape: long and wide

Handbook § 5.4

Long

nquest	year	y
173	2010	y_1
173	2012	y_2
…	…	…
173	2022	y_6



Wide

nquest	y2010	y2012	…	y2022
173	y_1	y_2	…	y_6

```
reshape wide y, i(nquest) j(year)   // i: new rows; j: suffix
reshape long                        // go back
```

Every other variable must be constant within `i`, or `reshape` stops.

collapse and loops

Handbook § 5.5, 12.1, 12.2

```
preserve
collapse (mean) y (median) y_median = y [pw = peso], by(year area3)
restore

foreach v of varlist y w c {          // ‘v’ takes the values y, w, c
    generate ‘v’_real = ‘v’ * rival
}
```

- `collapse` replaces the data with one row per group: use `preserve`/`restore`
- `[pw = peso]`: group means representative of the population
- `$name` recalls a global; `‘name’` (backtick, apostrophe) a local
- Run a loop as a whole block

Commands of the day

`generate`, `replace`	new variables	`merge 1:1`, `m:1`	add columns
`recode`	groups	`append`	add rows
`label values ... yesno`	dummies	`reshape wide/long`	change shape
`bysort`, `_n`, `_N`	groups	`collapse`	group statistics
`egen total/mean/max`	over many rows	`preserve`, `restore`	snapshot
`assert`	check	`local`, `foreach`	loops

PART 2

Now let's practice on Stata

Open `L2_exercises.do`, Part 2: five exercises, 10 minutes each,
then we write the solution together.

Before Lecture 3

Run `L2_solutions.do` (published after this lecture). It rebuilds from the raw data the files that Lecture 3 starts from. If you missed earlier lectures, run all the solutions do-files from Lecture 1, in order.